

VL-JEPA: Joint Embedding Predictive Architecture for Vision-language

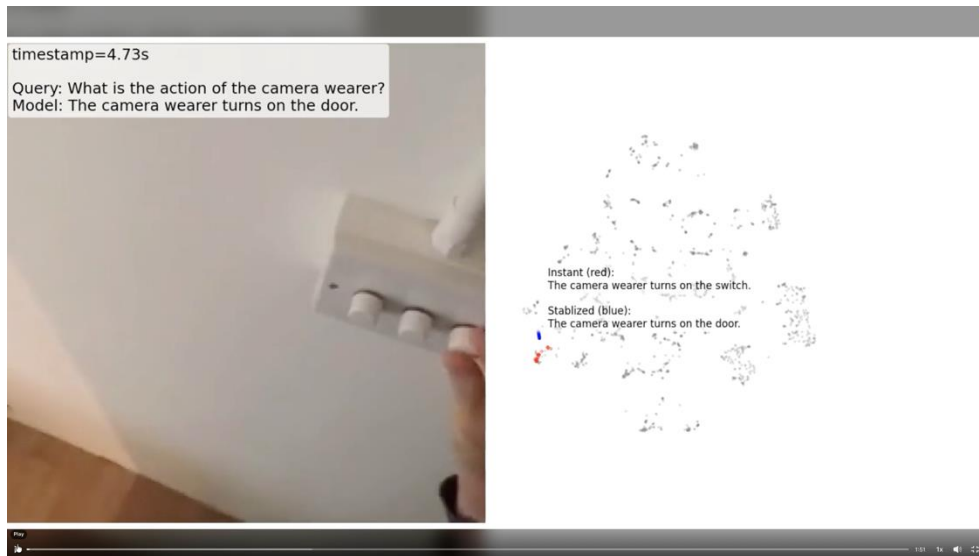
Delong Chen^{1,2,*} Mustafa Shukor^{1,3,*} Théo Moutakanni^{1,*} Willy Chung^{1,3,*} Jade Yu¹ , Tejaswi Kasarla¹ , Allen Bolourchi¹ , Yann LeCun^{1,4}, Pascale Fung^{1,2}

1 Meta FAIR 2 HKUST 3 Sorbonne Université 4 NYU

Presenter : Junghwan Yim

Abstract

- Predicts continuous text embeddings, abstracting superficial linguistic variance.
- Achieves superior performance with 50% fewer trainable parameters.
- Selective Decoding reduces the number of trainable parameters by approximately 2.85 times.
- Supports open-vocabulary classification, retrieval, and discriminative VQA.



Introduction

- One of the most important aspects of advanced artificial intelligence is the ability to understand the physical world around us. This capability allows AI systems to learn, reason, plan, and act in the real world to assist humans [LeCun, 2022].
- A common approach is to use large token-generating Vision-Language Models (VLMs) [Liu et al., 2023, Dai et al., 2023, Alayrac et al., 2022, Chen et al., 2024b, Cho et al., 2025, Chen et al., 2022].
- They take visual input X_V and a text query X_Q as input and autoregressively generate the desired text response Y in the token space, i.e., $(X_V, X_Q) \rightarrow Y$.
- Simple but insufficient for two main reasons:
 1. **First, VLMs are expensive to develop.** This is because VLMs are trained to capture not only the task-relevant semantics but also task-irrelevant superficial linguistic features, such as word choice, style, or paraphrasing, to generate the response Y to a query. During the training process, VLMs must model both of these aspects, ultimately consuming unnecessary computational effort in generating diverse token sequences that do not affect the accuracy of the output.
 2. **Second, real-time tasks using real-time streaming video (e.g., live action tracking) require sparse and selective decoding (e.g., outputting a description only when a new event occurs) [Zhou et al., 2024].** However, VLMs rely on autoregressive token-by-token decoding, which must be completed before the underlying meaning of Y is revealed. This process causes unnecessary latency and hinders the ability to dynamically update meaning in real time.

Introduction

- In this paper, we introduce the Vision-Language Joint Embedding Predictive Architecture (VL-JEPA).
- It switches the costly learning of data space token generation to more efficient latent space semantic prediction.
- As shown in Figure 1, the model uses an x-encoder to map the visual input X_V to an embedding S_V , and a y-encoder to map the text target Y to an embedding S_Y , and a predictor learns the mapping $(S_V, X_Q) \mapsto S_Y$, where X_Q is the text query (i.e., prompt).
- The learning objective is defined in the embedding space $\mathcal{L}_{VL-JEPA} = D(\hat{S}_Y, S_Y)$ instead of the data space $\mathcal{L}_{VLM} = D(Y, \hat{Y})$.
- During inference, a y-decoder converts the predicted embedding $\hat{S}_Y$ to the text space $\hat{Y}$ as needed.

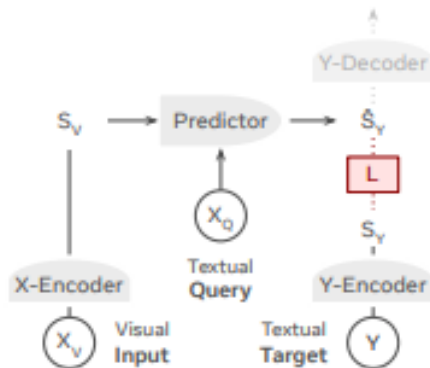


Figure 1. VL-JEPA model architecture

Introduction

VL-JEPA is not a generative model, so it does not need to reconstruct all superficial details of Y in the token space. Instead, it only predicts the abstract representation SY in the embedding space.

- In the embedding space, these diverse goals can be mapped to nearby points that share similar meanings. This simplifies the target distribution, making the learning process more efficient.
- Unlike existing VLMs, this approach eliminates the need to learn language generation using a heavy decoder during the training process, resulting in significant efficiency improvements.

Due to its non-autoregressive nature, VL-JEPA can generate a continuous stream of target semantic embeddings with minimal latency within a sliding window using only a single forward pass, without autoregressive decoding.

- Particularly advantageous for real-time online applications
- Selectively decoding the embedding stream with a lightweight y -decoder enables efficient and rapid updates.

Introduction

Existing token-generating VLMs [Liu et al., 2023, Cho et al., 2025] and strictly controlled comparison experiments Under the same training conditions, VL-JEPA consistently shows higher performance in zero-shot captioning and classification while using only about half of the trainable parameters.

Embedding space supervision improves training efficiency.

VL-JEPA significantly improves inference time efficiency through selective decoding.

Decoding occurs only when there is a significant change in the predicted embedding stream.

Reduces the number of decoding operations by about 2.85 times while maintaining overall output quality.

1. Pre-training stage to build strong vision-language alignment using caption data
 - VL-JEPABASE evaluates zero-shot classification and text-video retrieval performance.
 - Outperforms models like CLIP [Radford et al., 2021], SigLIP2 [Tschannen et al., 2025], and Perception Encoder [Bolya et al., 2025].
2. Supervised fine-tuning (SFT) stage that imbues the model with VQA (Visual Question Answering) capabilities.
 - VL-JEPASFT significantly improves classification performance by being exposed to in-domain training data.
 - Achieves performance comparable to existing VLM families such as InstructBLIP [Dai et al., 2023] and Qwen-VL [Bai et al., 2023].

Introduction

Summary

- We introduce VL-JEPA, the first non-generative model built upon the Joint Embedding Predictive Architecture that can perform general-domain vision-language tasks in real-time.
- Through controlled experiments, we demonstrate that VL-JEPA, trained with latent space embedding prediction, shows superior performance compared to VLMs relying on data space token prediction.
- We show that VL-JEPA offers significantly higher efficiency than VLMs in online video streaming applications, thanks to its non-autoregressive design and native support for selective decoding.
- We emphasize that our unified model architecture, VL-JEPASFT, can effectively handle a wide range of classification, retrieval, and Visual Question Answering (VQA) tasks simultaneously.

Methodology

Dataset

VL-JEPA is trained on a dataset composed of **three elements in the form of $\langle X_V, X_Q, Y \rangle$** . Here, X_V is the visual input (a single image or a sequence of video frames), X_Q is the text query (i.e., the question), and Y is the text target to be predicted (i.e., the answer).

Components

- 1. X-encoder ($X_V \mapsto S_V$):**
Compresses the large-volume visual input into concise visual embeddings, S_V , which are continuous vector sequences similar to the "visual tokens" of a classic VLM (Visual Language Model).
- 2. Predictor ($\langle S_V, X_Q \rangle \mapsto \hat{S}_Y$):**
The core component of VL-JEPA. It maps the visual embedding to the predicted value of the target embedding, $\hat{S}_Y$, conditioned on the text query.
- 3. Y-encoder ($Y \mapsto S_Y$):**
Embeds the text target, Y , into a continuous latent space, S_Y , which serves as the prediction target. The target embedding plays the role of abstracting away task-irrelevant information.
- 4. Y-decoder ($\hat{S}_Y \mapsto \hat{Y}$):**
Not used in the main training phase of VL-JEPA. During the inference phase, it converts the predicted embedding, $\hat{S}_Y$, into human-readable text, $\hat{Y}$, as needed.

Methodology

VL-JEPA Architecture

- The X-encoder is chosen as **V-JEPA 2 [Assran et al., 2025]**, a Vision Transformer that outputs visual token sequences.
- These tokens are projected and input into the Predictor, which is initialized using **Llama 3 Transformer layers**.
- Query conditioning is achieved by tokenizing and embedding the text query, then inputting the generated text token embeddings along with the visual embeddings into the Predictor.
- The output of the Llama 3 Transformer layers is pooled and projected into the target embedding space created by the Y-encoder. The Y-encoder is initialized with **EmbeddingGemma-300M [Vera et al., 2025]**.

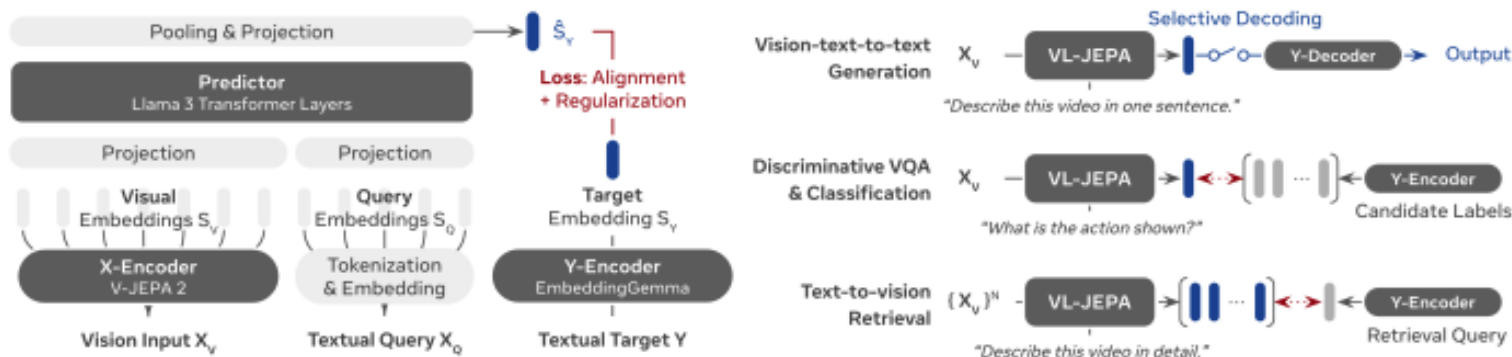


Figure 2. Left: VL-JEPA Architecture. It learns to predict the target embedding S_y , instead of reconstructing the raw target Y in token space as in classical VLMs. **Right: VL-JEPA Applications:** It handles vision-text-to-text generation tasks (e.g., captioning) with selective decoding mechanism natively supported. Furthermore, VL-JEPA's embedding space facilitates discriminative VQA, open-vocabulary classification and text-to-video retrieval tasks using a single unified model architecture.

Methodology

Training Objective

- 1) Prediction error in the embedding space, and
 - 2) Additional regularization to prevent representation collapse [Bardes et al., 2021, Balestrierio and LeCun, 2025].
- A loss function that satisfies both can be applied to VL-JEPA. The regularization term can be replaced with other representation collapse prevention strategies, such as using Exponential Moving Average (EMA) on the Y-encoder [Assran et al., 2025] or fixing the Y-encoder [Zhou et al., 2025].
 - In this study, the established InfoNCE loss function [Radford et al., 2021] in the vision-language domain is adopted.
 - Mathematically, the InfoNCE loss function [Wang and Isola, 2020] can be divided into: 1) a representation alignment term that minimizes the distance between the normalized predicted value and the target embedding, and 2) a uniformity regularization term that prevents representation collapse by pushing embeddings in a batch away from each other.
 - In this study, the Predictor and Y-encoder are jointly trained using a bidirectional InfoNCE loss function so that the two models can interact.

Methodology

Embedding Space

Compared to the token space loss used in generative VLMs, calculating the learning loss in the embedding space is advantageous because the target distribution is simplified.

In particular, many real-world prediction tasks are inherently ill-defined:

- Multiple plausible targets Y can exist for the same input X .
- However, in the raw one-hot token space, two sequences are orthogonal because they share no overlapping tokens.
- But when VL-JEPA's Y-encoder embeds these into nearby points (ideally creating a dense, unimodal distribution), the learning task becomes much easier.

The model no longer has to align with multiple separate, high-density regions in a sparse token space, but only with a single, consistent mode in the continuous embedding space.

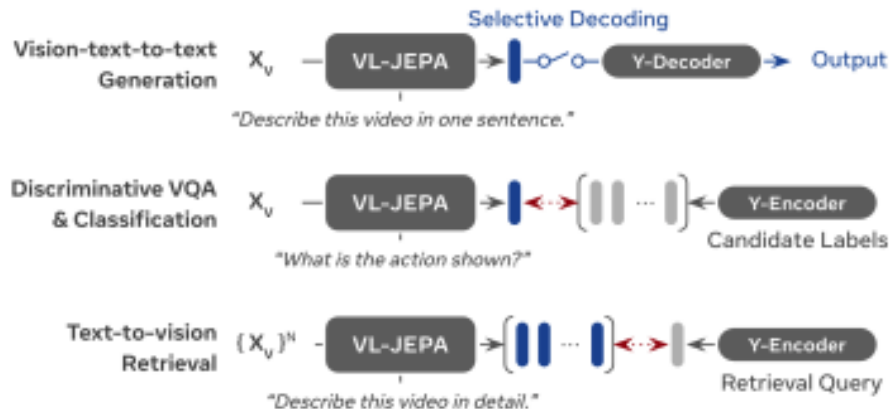
Methodology

Multiple Tasks

For vision-text-text generation tasks such as captioning or open-ended VQA, the query X_Q is a captioning prompt or a question, and the predictor is trained to predict the embedding $\hat{S}_Y$ of the target output, which is then decoded into text.

VL-JEPA also supports CLIP-style open-vocabulary classification and discriminative VQA. Here, candidate label texts are encoded into embeddings and the closest match is selected by comparing them with the prediction $\hat{S}_Y$.

For text-video retrieval, candidate videos are mapped to the prediction embedding $\hat{S}_Y$ using a retrieval captioning prompt, and then ranked according to their similarity with the encoded text search query.



Methodology

Selective Decoding

Real-world video applications often require online streaming inference. The core challenge is balancing two conflicting requirements: the model must continuously update semantic information as new frames arrive, but computational efficiency and latency are also crucial.

Existing Visual Language Models (VLMs)

The costly nature of continuously running autoregressive language models means they:

- Utilize explicit memory mechanisms [Zhou et al., 2024, Qian et al., 2024] to decide when to decode.
- Or, rely on complex KV-cache optimizations [Di et al., 2025] for efficiency.

VL-JEPA natively supports Selective Decoding

Because it predicts semantic answer embeddings in a non-autoregressive manner, the model provides a continuous semantic stream $\hat{S}_Y$ that can be monitored in real-time.

- This stream can be stabilized through simple smoothing (e.g., average pooling).
- Decoding is performed only when a significant semantic change is detected, such as the local window variance exceeding a threshold.

=> This achieves both responsiveness and efficiency by avoiding unnecessary decoding while maintaining constant semantic monitoring.

Implementation

Model Architecture

X-encoder

- Uses V-JEPA 2 ViT-L [Assran et al., 2025] with 304 million parameters.
- A self-supervised vision model that shows excellent performance in both image and video tasks.
- Each video input is uniformly sampled as frames with 256x256 resolution.
- For image input, the same image is duplicated to match the input shape.

Predictor

- Initialized with the last 8 Transformer layers of Llama-3.2-1B, with 490 million trainable parameters.
- The text tokenizer and token embedding are also from Llama-3.2-1B.
- Accepts a maximum of 512 query tokens, using [PAD] tokens for short queries.
- The causal attention mask is disabled to allow joint attention over both visual and query embeddings.
- A linear projection connects the Predictor to the visual and text embeddings, and mean pooling is applied over non-[PAD] tokens to obtain the predicted target embedding.

Y-encoder

- Initialized with EmbeddingGemma-300M [Vera et al., 2025].
- The maximum context length is set to 512 to handle detailed captions.
- Setting a learning rate multiplier of 0.05 on all text encoder parameters improves performance, as the quality of embedding prediction is not optimized during the early training stages.
- A shared embedding space of 1,536 dimensions is obtained by applying a linear projection head to both the Predictor and Y-encoder, and the loss is calculated in this space.

Implementation

Large-scale Pre-training

The first query-less pre-training stage aims to build strong vision-language alignment using large-scale caption data.

- **Image-text data:** PLM-Image-Auto [Cho et al., 2025], Datacomp [Gadre et al., 2023], and YFCC-100M [Thomee et al., 2016].
- **Video-text data:** Action100M, an internal dataset composed of captions generated from PLM-Video-Auto [Cho et al., 2025], Ego4D Atomic Action Descriptions [Grauman et al., 2022], and HowTo100M videos [Chen et al., 2025b].
 1. First, image-only training is performed using only one frame per visual input on the Datacomp and YFCC 100M datasets with a large batch size of 24k.
=> After 100,000 iterations of training, the model learned 2 billion samples and achieved an ImageNet zero-shot accuracy of 61.6%.
 2. Then, image-video joint pre-training is conducted using 16 frames per input.

Configuration:

- Pre-training was conducted for two weeks using 24 nodes, each equipped with 8 NVIDIA H200 GPUs.
- A constant learning rate of 5×10^{-5} was used for long-term training.

=> VL-JEPABASE (used to measure zero-shot classification and retrieval performance).

Implementation

Query-Conditional Supervised Fine-Tuning (SFT) Stage

This stage aims to improve the VQA capabilities of VL-JEPA while preserving the pre-trained vision–language alignment used for classification and retrieval.

Dataset

The training data is selected from the PLM data mixture [Cho et al., 2025] and consists of:

- 25 million VQA samples,
- 2.8 million caption generation samples,
- 1.8 million classification samples, and
- a downsampled subset of data from the pre-training stage to mitigate catastrophic forgetting.

Configuration

Training is conducted for 35,000 steps with a batch size of 6,000 (approximately 2 days on 24 nodes), and a cosine learning rate decay schedule is applied to improve convergence.

=> VL-JEPASFT Model

- evaluated VQA functionality
- compared against state-of-the-art
- zero-shot evaluation was not performed

Experiments

4.1 Classification and Retrieval

VL-JEPA is evaluated on an extensive set of benchmarks, including 8 classification datasets and 8 retrieval datasets. In zero-shot evaluation, it is compared with general-purpose base models such as

- CLIP [Radford et al., 2021],
- SigLIP2 [Tschannen et al., 2025], and
- Perception Encoder (PE-Core) [Bolya et al., 2025].

Table 1. Video classification and text-to-video retrieval. Best zero-shot performance in each dataset are highlighted. Samples seen = training step $\times$ effective batch size.

Model		# Parameters	# Samples Seen	Zero-shot	Generalist Model	Video Classification (Top-1 Accuracy)								Text-to-video Retrieval (Recall@1)									
						Average	SfV2	EK100	EgoExo4D	Kinetics-400	COIN (SR)	COIN (TR)	CrossTask (SR)	CrossTask (TR)	Average	MSR-VTT	ActivityNet	DeDeMo	MSVD	YouCook2	PVD-Bench	Dream-1k	VDC-1k
CLIP	RN50	75M	12.8B			21.8	2.1	1.5	1.9	41.4	8.6	39.0	10.9	68.7	28.3	28.7	17.7	24.7	29.7	5.1	27.6	47.2	46.0
	ViT-B	124M	12.8B	✓	✓	25.3	3.1	1.3	2.4	49.5	11.2	47.3	16.2	71.5	29.3	31.0	19.5	25.7	34.0	6.1	27.0	48.5	42.9
	ViT-L	389M	12.8B			30.9	3.8	3.7	3.6	58.3	14.7	63.5	20.8	78.5	35.3	35.9	23.4	30.7	41.9	7.9	36.7	56.8	49.3
SigLIP2	ViT-B	375M	40B			33.9	5.2	2.3	4.9	57.8	20.6	69.9	27.7	82.9	39.6	40.2	25.0	32.1	48.6	13.8	52.1	60.9	43.7
	ViT-L	882M	40B	✓	✓	38.7	5.9	4.5	7.0	63.6	24.2	78.5	35.1	90.8	45.4	41.6	32.7	35.1	53.5	19.0	59.2	71.6	50.9
	ViT-g	1.9B	40B			39.9	6.1	6.1	6.4	68.0	26.0	80.4	35.1	90.8	47.5	43.4	33.9	38.9	56.0	22.2	60.4	73.0	52.5
PE-Core	ViT-B	448M	58B			37.3	5.8	3.3	6.3	65.4	21.5	77.1	26.9	91.8	44.9	46.5	35.4	35.3	49.1	15.2	59.8	68.7	49.2
	ViT-L	671M	58B	✓	✓	42.8	9.3	6.0	10.9	73.4	27.1	83.3	37.5	95.3	50.2	48.9	41.7	40.8	56.2	22.5	64.7	75.9	51.0
	ViT-G	2.3B	86B			44.6	9.0	6.4	13.0	76.4	29.0	86.0	40.3	97.2	58.1	51.6	49.1	44.5	58.7	26.0	77.0	89.2	68.5
VL-JEPA _{BASE}	ViT-L	1.6B	2.0B	✓	✓	46.4	16.1	13.3	21.1	57.8	39.8	74.4	60.5	88.0	58.4	37.6	55.4	49.2	47.9	23.1	78.2	88.8	87.2
VL-JEPA _{SFT}	ViT-L	1.6B	2.5B	✗	✓	70.7	68.2	38.8	59.5	81.4	60.3	86.8	77.1	93.0	59.5	43.7	53.8	46.2	49.1	28.8	81.1	86.4	86.7
SoTA (including specialist models)				✗	✗	-	77.5	56.4	47.8	92.1	67.3	95.3	64.5	96.0	-	62.8	74.1	74.2	61.4	28.9	77.0	89.2	68.5

Experiments

4.2 Visual Question Answering

We evaluate VL-JEPASFT on the discriminative VQA task. The inference process selects the answer that minimizes the distance to the predicted embedding by encoding candidate answers using the Y-encoder. Four benchmarks prioritize visual perception over knowledge or reasoning:

- **GQA:** Accuracy on the testdev-balanced split for the GQA dataset, a real-world visual reasoning and compositional question-answering dataset [Hudson and Manning, 2019].
- **TallyQA:** Weighted average accuracy on the "simple" and "complex" splits for TallyQA [Acharya et al., 2019], following the method of Chen et al. [2022], which aims at complex counting.
- **POPE:** Average accuracy on the "random," "popular," and "adversarial" settings on the MS-COCO dataset for POPE, to evaluate object hallucination.

Table 2. VQA benchmarks. We report accuracy on GQA [Hudson and Manning, 2019], TallyQA [Acharya et al., 2019], POPE [Li et al., 2023b], and POPEv2 [Li et al., 2025b]. Scores lower than our model are marked in **red**. Scores from SmolVLM are obtained by our evaluation, while other baselines are reported in the literature.

<i>GQA: compositional visual reasoning</i>		<i>TallyQA: complex object counting</i>		<i>POPE: object hallucination</i>		<i>POPEv2: object hallucination</i>	
Model	Accuracy	Model	Accuracy	Model	Accuracy	Model	Accuracy
BLIP-2 (OPT-2.7B)	33.9	SmolVLM-256M	32.3	SmolVLM2-256M	56.4	SmolVLM-256M	62.3
BLIP-2 (FlanT5XXL)	41.0	SmolVLM-500M	44.8	SmolVLM-256M	57.9	LLaVA-1.5-13B	72.7
InstructBLIP (FlanT5XL)	48.4	PaLI-700M	62.3	LLaVA-7B	72.9	InternVL2-8B	74.5
InstructBLIP (Vicuna-13B)	49.5	SmolVLM-2B	64.7	InstructBLIP (Vicuna-13B)	79.0	InternVL2-26B	76.1
Qwen-VL-Chat-7B	57.5	PaLI-3B	65.8	Video-LLaVA (7B)	83.4	Qwen2-VL-72B	79.4
Qwen-VL-7B	59.3	InstructBLIP (Vicuna-13B)	68.0	SmolVLM-500M	85.8	SmolVLM-500M	83.8
InternVL-Chat (Vicuna-7B)	59.5	PaLI-17B	71.9	LLaVA-1.5-7B	85.9	Qwen2-VL-7B	87.0
LLaVA-1.5 (Vicuna-7B)	62.0	LLaVA-1.5 (Vicuna-13B)	72.3	LLaVA-1.5-13B-HD	86.3	SmolVLM-2B	88.8
InternVL-Chat (Vicuna-13B)	66.6	PaliGemma (3B)	76.8	SmolVLM-2B	87.5	Qwen2-VL-2B	91.3
VL-JEPASFT (1.6B)	60.8	VL-JEPASFT (1.6B)	67.4	VL-JEPASFT (1.6B)	84.2	VL-JEPASFT (1.6B)	82.2

Experiments

4.3 WorldPrediction-WM

VL-JEPA is evaluated on the "World Modeling" task of the WorldPrediction [Chen et al., 2025a] benchmark. In this task, the model is provided with two images representing the initial and final world states, and it must identify the action that describes the observed state change from four candidate video clips.

To apply VL-JEPA, the initial and final state images are duplicated and concatenated to extract a state embedding. Each action candidate is then encoded into an action embedding. The model then selects the candidate whose embedding is closest to the state embedding.

Table 3. WORLDPrediction-WM benchmark results. We compare the accuracy between large VLMs, socratic LLMs, and VL-JEPA. VL-JEPA_{SFT} achieves a new SoTA at 65.7%.

Vision Language Models								Socratic LLMs (w/ Qwen2.5-VL-72B captions)										VL-JEPA	
InternVL2.5				Qwen2.5-VL				Llama-3.1		Llama-4		Qwen2.5		GPT-4o	Claude-3.5	Gemini-2	BASE	SFT	
2B	4B	26B	38B	3B	7B	32B	72B	8B	70B	109B	400B	3B	7B	72B	N/A	N/A	N/A	1.6B	1.6B
20.0	29.8	30.2	50.3	21.6	45.5	49.0	57.0	48.7	49.8	52.7	53.6	44.0	49.1	48.5	52.0	53.3	55.6	63.9	<u>65.7</u>

Experiments

4.4 Embedding Prediction vs. Token Prediction: A Controlled Comparison

In this section, VL-JEPA is compared with a token-generating VLM benchmark under strictly identical training conditions. The same Perception Encoder [Bolya et al., 2025] (fixed ViT-L-14, 336x336 resolution, no tiling, 16 frames per video) is used for visual input. The same number of training iterations, the same effective batch size of 128, the same learning rate scheduler, and the same pre-training data mixture (§3) as previously described are used.

VL-JEPA predicts the target embedding [Duquenne et al., 2023] using a 0.5B predictor. In contrast, the VLM benchmark predicts the next token via cross-entropy using a 1B LLM.

For the VLM, the standard training method and codebase of PerceptionLM [Cho et al., 2025] are used to align the fixed vision encoder with the text-only LLM Llama-3.2-1B. For VL-JEPA, the predictor is initialized with the 8th to 16th layers of Llama-3.2-1B.

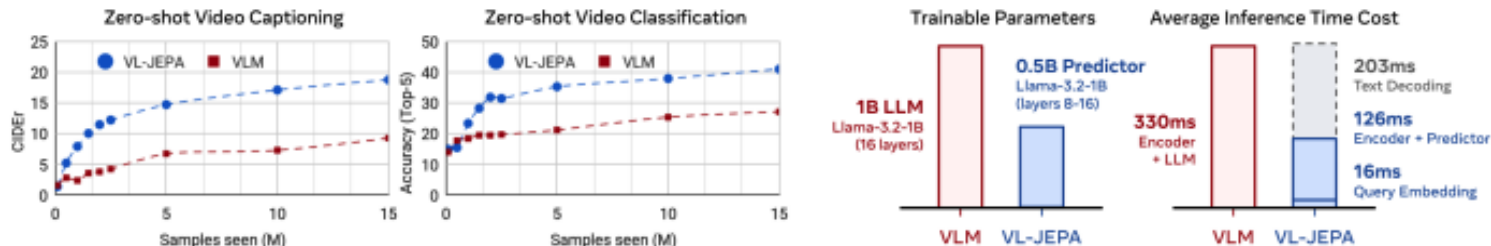


Figure 3. Comparison of embedding prediction (VL-JEPA) and token prediction (VLM). We conduct a fair comparison of under strictly aligned training settings (encoder, data, batchsize, etc.). **Left:** Zero-shot video captioning CIDEr score averaged over 3 datasets and zero-shot classification accuracy (top-5) averaged over 3 benchmarks. **Right:** Comparing the trainable parameters and average inference time cost.

Experiments

4.5 Effectiveness of Selective Decoding

We evaluate the effectiveness of VL-JEPA's embedding-based selective decoding on long video streams.

We design a benchmark task that minimizes the number of text decoding operations while recovering the temporally ordered annotation sequence.

- Decoding is performed only at selected points along the VL-JEPA embedding stream to generate a sequence of N decoded outputs $[(\hat{t}_1, \hat{y}_1), (\hat{t}_2, \hat{y}_2), \dots, (\hat{t}_N, \hat{y}_N)]$.
- Each ground-truth annotation $[(t_1, y_1), (t_2, y_2), \dots, (t_T, y_T)]$ is matched with the temporally closest decoded output.
- CIDEr scores are calculated for the matched pairs.

We used the EgoExo4D [Grauman et al., 2024] validation set.

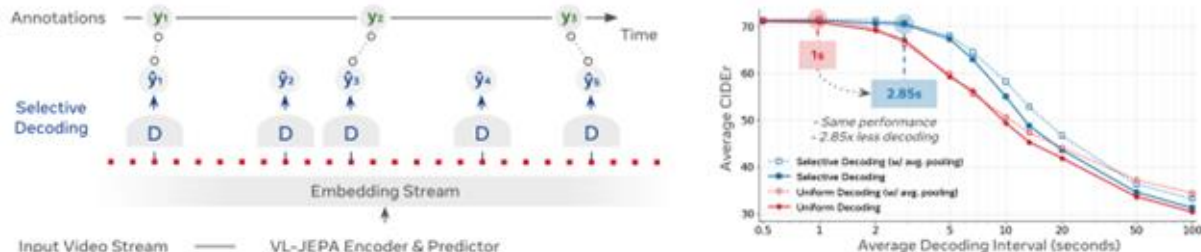


Figure 4. Evaluation of selective decoding. Left: We compare uniform sampling of decoding points at fixed intervals (red) and embedding-guided selective decoding (blue). Performance is measured by the average CIDEr score between each annotation y and its closest decoded output $\hat{y}$. Right: Results on EgoExo4D show that selective decoding achieves a Pareto improvement over uniform sampling: for the same performance level, it requires fewer decoding operations.

Experiments

4.6 Evaluation of Y-Encoder

We evaluate whether the JEPA architecture improves Y-encoder performance following a single-modal Text-Only (TOT) evaluation setup.

Hard negative benchmarks used:

- SugarCrep++ [Dumpala et al., 2024a]
- VISLA [Dumpala et al., 2024b]

These test the sensitivity to semantic and lexical changes in image descriptions. The setup includes two semantically similar captions (p_1 and p_2) for the same image and one negative caption (n) generated by changing an attribute, relation, or object. We compare the Y-encoders of different models by calculating the cosine similarity for all caption pairs. We check if the similarity between the positive captions, $\text{sim}(p_1, p_2)$, is higher than the similarity between each positive caption and the negative caption, $\text{sim}(p_1, n)$ and $\text{sim}(p_2, n)$. We report the accuracy (%) across all samples.

Table 4. Comparison of text-encoders performance. We report triplet-based accuracy (%) on SugarCrep++ and VISLA datasets.

Model		# Params. (total)	# Params. (text encoder)	SugarCrep++ [Dumpala et al., 2024a]						VISLA [Dumpala et al., 2024b]		
				Average	Replace Attribute	Replace Object	Replace Relation	Swap Attribute	Swap Object	Average	Generic	Spatial
CLIP	ViT-L	389M	85M	44.5	56.7	83.0	42.5	27.0	13.5	34.5	37.6	31.3
SigLIP2	ViT-g	1.9B	708M	56.5	66.9	74.4	<u>52.1</u>	<u>58.4</u>	<u>30.6</u>	<u>40.4</u>	<u>48.7</u>	32.1
PE-Core	ViT-G	2.3B	537M	<u>58.6</u>	<u>73.6</u>	90.6	48.9	53.2	26.5	38.3	45.2	31.4
VL-JEPA_{BASE}	ViT-L	1.6B	300M	63.9	<u>72.2</u>	<u>90.1</u>	52.2	62.9	42.0	42.9	49.8	35.9
VL-JEPA_{SFT}	ViT-L	1.6B	300M	58.4	68.5	90.9	47.4	55.4	29.8	39.5	44.8	<u>34.2</u>

Experiments

4.7 Ablation Study

Trained for 10,000 steps with a batch size of 512 (5 million samples) and a fixed learning rate using the SFT stage dataset

Table 5. Ablation studies results. The default setting adopted by VL-JEPA is marked in blue. We calculate $\pm\delta$ within each group of ablations in comparison with the default setting.

	Classification (Accuracy)	Retrieval (Recall@1)	VQA (Accuracy)
VL-JEPA _{SFT}	59.1	70.6	53.2
(a) Effectiveness of pretraining stage on caption data			
w/ Pretraining	49.0	47.5	46.1
w/o Pretraining	27.3 (-21.7)	30.2 (-17.3)	42.5 (-3.6)
(b) Learning rate multiplier for Y-Encoder			
multiplier = 0.05	27.3	30.2	42.5
multiplier = 1.00	23.7 (-3.6)	28.8 (-1.4)	40.7 (-1.8)
multiplier = 0.10	26.9 (-0.4)	30.2 (-0.0)	42.9 (+0.4)
multiplier = 0.01	25.6 (-1.7)	27.7 (-2.5)	41.0 (-1.5)
multiplier = 0.00	20.0 (-7.3)	25.9 (-4.3)	41.4 (-1.1)
(c) Loss function (with no projection head on top frozen text encoder)			
InfoNCE	23.3	30.3	44.3
Cosine	16.5 (-6.8)	20.2 (-10.1)	46.6 (+2.3)
L1	14.8 (-8.5)	15.5 (-14.8)	41.9 (-2.4)
L2	13.5 (-9.8)	11.7 (-18.6)	43.7 (-0.6)

	Classification (Accuracy)	Retrieval (Recall@1)	VQA (Accuracy)
(d) Predictor architecture and initialization			
Layer 8-16	27.3	30.2	42.5
Layer 0-2	24.3 (-3.0)	27.8 (-2.4)	40.1 (-2.4)
Layer 0-4	25.1 (-2.2)	28.9 (-1.3)	43.6 (+1.1)
Layer 0-8	27.2 (-0.1)	29.3 (-0.9)	43.4 (+0.9)
Layer 0-16	27.4 (+0.1)	31.0 (+0.8)	45.5 (+3.0)
w/o Bi-direction Attention	26.7 (-0.6)	31.2 (+1.0)	40.6 (-1.9)
w/o Llama-3 Initialization	28.1 (+0.8)	30.4 (+0.2)	40.6 (-1.9)
(e) Y-Encoder (trainable linear projection on top of frozen text encoder)			
EmbeddingGemma-300M	19.5	24.1	42.5
Qwen3-Embedding-0.6B	24.5 (+5.0)	24.5 (+0.4)	41.5 (-1.0)
Qwen3-Embedding-4B	27.7 (+8.2)	26.6 (+2.5)	38.1 (-4.4)
Qwen3-Embedding-8B	29.6 (+10.1)	29.5 (+5.4)	41.9 (-0.6)
PE _{core} -B (356M)	29.4 (+9.9)	34.5 (+10.4)	35.9 (-6.6)
PE _{core} -L (356M)	29.0 (+9.5)	34.2 (+10.1)	42.9 (+0.4)
PE _{core} -G (539M)	33.9 (+14.4)	32.0 (+7.9)	41.8 (-0.7)

Conclusion

Summary

- We present VL-JEPA, a novel Vision-Language model built upon the Joint Embedding Predictive Architecture.
- VL-JEPA simplifies the learning objective by switching the supervised learning target from the discrete token space to the continuous semantic embedding space, avoiding unnecessary modeling of superficial linguistic variance and enabling non-autoregressive prediction.
- Through controlled experiments, VL-JEPA demonstrates superior performance compared to token-generating Vision-Language Models trained with cross-entropy loss under the same training data budget, and proves to be significantly more efficient in terms of training efficiency and inference latency.
- Beyond generative tasks, its embedding-based design allows VL-JEPA to handle open-vocabulary classification and cross-modal retrieval within a single unified architecture. Furthermore, its ability to generate a continuous stream of semantic embeddings makes it particularly suitable for real-time video applications, where selective decoding enhances both responsiveness and efficiency.

Limitation

The goal is not to present a universal alternative to Vision-Language Models, as this would require a more extensive evaluation on tasks where current token-generating Vision-Language Models excel, such as reasoning, tool use, and agent behavior.